

Terraform MCP Server: AI-Assisted Infrastructure as Code

Purpose

Terraform MCP connects AI tools with Terraform tools and information, helping developers create and understand Terraform configurations faster.

Context

Terraform development can be complex when working with different cloud resources, providers, and configurations. Terraform MCP gives AI tools a Terraform-specific context to help generate and understand the required code.

Real-World Example

A DevOps engineer needs to create a secure AWS S3 bucket using Terraform. The engineer asks an AI tool to create the Terraform configuration. Terraform MCP provides Terraform-specific information, helping the AI generate the required configuration.

Problem

AI tools may generate outdated, deprecated, or incorrect Terraform configurations. Developers may also need to manually search Terraform documentation and verify resource and provider details.

Problem Solution

Utilize an AI tool and Terraform MCP to minimize manual Terraform documentation searching, repetitive coding, and configuration errors.

AI Tool → Terraform MCP → Terraform Code → Terraform CLI → Validate → Plan → Human Approval → Apply

Business Outcome

- Faster infrastructure development
- Reduced Terraform configuration errors
- Improved DevOps productivity
- More consistent Infrastructure as Code
- Safer production changes with human approval

NOTE

Terraform MCP does not replace Terraform CLI and does not automatically mean Terraform apply. Terraform MCP assists the AI; Terraform CLI manages the infrastructure. Install steps
last page

Step 1: install geeminu cli and pull Docker Terraform mcp official image, check docker ps

docker run -d --name terraform-mcp --restart unless-stopped hashicorp/terraform-mcp-server

```
ubuntu@ip-172-31-36-199:~$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
c211afd03a07   hashicorp/terraform-mcp-server      "/bin/terraform-mcp-...  33 hours ago   Restarting (0) 8 seconds ago
               terraform-mcp
```

Command to check which MCP is connected: `gemini mcp list`

prompt: `Create Terraform code for a secure AWS S3 bucket with versioning and encryption`

```
ubuntu@ip-172-31-36-199:~$ gemini
 Gemini CLI v0.54.0 - Updating
Authenticated with gemini-api-key /auth

Tips for getting started:
1. Create GEMINI.md files to customize your interactions
2. /help for more information
3. Ask coding questions, edit code or run commands
4. Be specific for the best results
x Automatic update failed. Please try updating manually:

  npm install -g @google/gemini-cli@0.54.4

[!] Gemini CLI update available! 0.54.0 -> 0.54.4
    Installed with npm. Attempting to automatically update now...

> gemini mcp list

  ✓ List MCP Resources List MCP resources
Listed 2 resources.

> I will execute the gemini mcp list command in the shell to retrieve the list of configured MCP servers and their statuses from the Gemini CLI system.

  ✓ Shell gemini mcp list
Configured MCP servers:

  ✓ terraform: docker run -i --rm hashicorp/terraform-mcp-server (stdio) - Connected
  █

> The configured Model Context Protocol (MCP) servers in your Gemini CLI environment:

  1 ✓ terraform: docker run -i --rm hashicorp/terraform-mcp-server (stdio) - Connected

> Create Terraform code for a secure AWS S3 bucket with versioning and encryption

  ✓ FindFiles '**/GEMINI.md' - No files found
```

Step 2: Then it starts generating the code based on the prompt

(<https://docs.aws.amazon.com/AmazonS3/latest/userguide/default-s3-c-encryption-setting-faq.html>).

Example Usage

```
1 resource "aws_kms_key" "mykey" {
2   description = "This key is used to encrypt bucket objects"
3   deletion_window_in_days = 10
4 }
5
6 resource "aws_s3_bucket" "mybucket" {
7   bucket = "mybucket"
8 }
9
10 resource "aws_s3_bucket_server_side_encryption_configuration" "example" {
11   bucket = aws_s3_bucket.mybucket.id
12
13   rule {
14     apply_server_side_encryption_by_default {
15       kms_master_key_id = aws_kms_key.mykey.arn
16       sse_algorithm     = "aws:kms"
17     }
18   }
19 }
```

Blocking SSE-C Uploads

```
1 resource "aws_kms_key" "mykey" {
2   description = "This key is used to encrypt bucket objects"
3   deletion_window_in_days = 10
4 }
5
6 resource "aws_s3_bucket" "mybucket" {
7   bucket = "mybucket"
8 }
9
10 resource "aws_s3_bucket_server_side_encryption_configuration" "example" {
11   bucket = aws_s3_bucket.mybucket.id
12
13   rule {
14     apply_server_side_encryption_by_default {
15       kms_master_key_id = aws_kms_key.mykey.arn
16       sse_algorithm     = "aws:kms"
17     }
18     bucket_key_enabled = true
19     blocked_encryption_types = ["SSE-C"]
20   }
21 }
```

Argument Reference

Step 3: It is asking for this saved file in specifi dir path

> Save this Terraform code as COTASK/secure_s3.tf

Creating COTASK/secure_s3.tf: Creating and validating the secure S3 bucket Terraform configuration file.

? WriteFile Writing to COTASK/secure_s3.tf

... first 69 lines hidden (Ctrl+O to show) ...

```
70     effect = "Deny"
71
72     principals {
73       type       = "*"
74       identifiers = ["*"]
75     }
76
77     actions = [
78       "s3:*"
79     ]
80
81     resources = [
82       aws_s3_bucket.secure_bucket.arn,
83       "${aws_s3_bucket.secure_bucket.arn}/*",
84     ]
85
86     condition {
87       test     = "Bool"
88       variable = "aws:SecureTransport"
89       values   = ["false"]
90     }
91   }
92 }
93
94 resource "aws_s3_bucket_policy" "secure_bucket_policy" {
95   bucket = aws_s3_bucket.secure_bucket.id
96   policy = data.aws_iam_policy_document.allow_ssl_only.json
97
98   # Ensure public access block and ownership controls are applied before policy attachment
99   depends_on = [
100     aws_s3_bucket_public_access_block.secure_bucket_public_access,
101     aws_s3_bucket_ownership_controls.secure_bucket_ownership
102   ]
103 }
```

Apply this change?

- 1. Allow once
- 2. Allow for this session
- 3. Modify with external editor

➤ **Now the file is successfully stored in the dir**

Topic: Completed the creation and styling of the secure S3 bucket configuration file.

I have successfully created and saved the secure AWS S3 bucket configuration to COTASK/secure_s3.tf.

File Summary: COTASK/secure_s3.tf

The file now contains the following secure configuration:

- **Versioning Enabled:** Protects against accidental deletion and permits object recovery.
- **Server-Side Encryption (SSE-S3):** Encrypts data at rest using S3 managed keys (AES256) and optimizes cost with bucket keys.
- **Public Access Block Configuration:** Blocks all public ACLs and public policies.
- **Ownership Controls:** Disables S3 Access Control Lists (ACLs) to enforce IAM-based bucket owner policies.
- **In-Transit Encryption:** Attaches a bucket policy that denies any unencrypted HTTP traffic (aws:SecureTransport = false).

? for shortcuts

auto-accept edits Shift+Tab to plan

1 MCP server

Step 4: Check that the folder directory is available and not ls -lhart show below image marked

```
ubuntu@ip-172-31-36-199:~/COTASK$ ls -la
total 244
drwxrwxr-x 27 ubuntu ubuntu 4096 Aug  7 17:30 .
drwxr-x--- 44 ubuntu ubuntu 4096 Aug  8 02:17 ..
drwxrwxr-x  8 ubuntu ubuntu 4096 Jul 26 16:12 .git
drwxrwxr-x  6 ubuntu ubuntu 4096 Jul 26 16:12 'AWS EC2,Lamda,RDS,IAM'
drwxrwxr-x  2 ubuntu ubuntu 4096 Jul 26 16:12 AWS-VPC-Landing-Zone-Terraform-main
drwxrwxr-x  2 ubuntu ubuntu 4096 Jul 26 16:12 AZ
drwxrwxr-x  4 ubuntu ubuntu 4096 Jul 26 16:12 CICD_BlueGreen_EC2-main
drwxrwxr-x  2 ubuntu ubuntu 4096 Jul 26 16:12 'Day 29 AzureDevOps-Part-2'
drwxrwxr-x  7 ubuntu ubuntu 4096 Jul 26 16:12 Docker-Compose-Examples-master
-rw-rw-r--  1 ubuntu ubuntu 4093 Jul 26 16:12 EC2podnewreagion.tf
-rw-rw-r--  1 ubuntu ubuntu 4166 Jul 26 16:12 EC2prod.tf
drwxrwxr-x  5 ubuntu ubuntu 4096 Jul 26 16:12 'EKS-Terraform-Github-Actions-master and Jenkins terraform module file and script'
drwxrwxr-x  2 ubuntu ubuntu 4096 Jul 26 16:12 'IAM user and polices'
-rw-rw-r--  1 ubuntu ubuntu 476 Jul 26 16:12 LB.tf
-rw-rw-r--  1 ubuntu ubuntu 1773 Jul 26 16:12 Main.tf
drwxrwxr-x 38 ubuntu ubuntu 4096 Jul 26 16:12 Mastering-DevSecOps-Master
-rw-rw-r--  1 ubuntu ubuntu 1880 Jul 26 16:12 README.md
-rw-rw-r--  1 ubuntu ubuntu  0 Jul 26 16:12 Redme.txt
drwxrwxr-x  4 ubuntu ubuntu 4096 Jul 26 16:12 Script
drwxrwxr-x  6 ubuntu ubuntu 4096 Jul 26 16:12 Terraform-for-AWS-master
drwxrwxr-x 23 ubuntu ubuntu 4096 Jul 26 16:12 TerraformScript-Co
drwxrwxr-x  4 ubuntu ubuntu 4096 Jul 26 16:12 azure-ci-cd-pipeline-main
-rw-rw-r--  1 ubuntu ubuntu 160 Jul 26 16:12 backend.tf
drwxrwxr-x  5 ubuntu ubuntu 4096 Jul 26 16:12 blue-green-deploy-on-kubernetes-cluster-master
drwxrwxr-x  3 ubuntu ubuntu 4096 Jul 26 16:12 compose-dev-env-main
-rw-rw-r--  1 ubuntu ubuntu  73 Jul 26 16:12 docker
-rw-rw-r--  1 ubuntu ubuntu 662 Jul 26 16:12 docker.txt
-rw-rw-r--  1 ubuntu ubuntu 239 Jul 26 16:12 ebs.tf
drwxrwxr-x  6 ubuntu ubuntu 4096 Jul 26 16:12 gitlabcid
-rw-rw-r--  1 ubuntu ubuntu 1930 Jul 26 16:12 install_jenkins.sh
drwxrwxr-x  5 ubuntu ubuntu 4096 Jul 26 16:12 'jenkins-DeployECS UAT-POD'
drwxrwxr-x  2 ubuntu ubuntu 4096 Jul 26 16:12 jenkins-deploy-eks-via-eksctl-master
-rw-rw-r--  1 ubuntu ubuntu  19 Jul 26 16:12 job
drwxrwxr-x  3 ubuntu ubuntu 4096 Jul 26 16:12 kubernetes-yaml-templates-master
-rw-rw-r--  1 ubuntu ubuntu 4116 Jul 26 16:12 main.tf
drwxrwxr-x  5 ubuntu ubuntu 4096 Jul 26 16:12 mongodb
drwxrwxr-x  3 ubuntu ubuntu 4096 Jul 26 16:12 'multi-cloud-setup-main Githubaction'
-rw-rw-r--  1 ubuntu ubuntu 30107 Jul 26 16:12 'multi-cloud-setup-main.zip'
-rw-rw-r--  1 ubuntu ubuntu 189 Jul 26 16:12 provider.tf
-rw-rw-r--  1 ubuntu ubuntu 10721 Jul 27 17:11 report.json
drwxrwxr-x  4 ubuntu ubuntu 4096 Jul 26 16:12 reportportal-master
-rw-rw-r--  1 ubuntu ubuntu 305 Jul 26 16:12 s3.tf
-rw-rw-r--  1 ubuntu ubuntu 3042 Aug  7 17:30 secure_s3.tf
-rw-rw-r--  1 ubuntu ubuntu 226 Jul 26 16:12 terraformmodule
-rw-rw-r--  1 ubuntu ubuntu 4110 Jul 26 16:12 'terraform EKS script.txt'
drwxrwxr-x  3 ubuntu ubuntu 4096 Jul 26 16:12 terraforme
drwxrwxr-x 13 ubuntu ubuntu 4096 Jul 26 16:12 'terraformmyt-main AZURE'
```

- Check inside folder and file **cat COTASK/secure_s3.tf**

```
ubuntu@ip-172-31-36-199:~$ cat COTASK/secure_s3.tf
# 1. The S3 Bucket Resource
resource "aws_s3_bucket" "secure_bucket" {
  bucket = "gamingproject-secure-bucket" # Must be globally unique
  force_destroy = false                 # Prevent accidental deletion of non-empty bucket

  tags = {
    Name      = "secure-production-bucket"
    Environment = "production"
    ManagedBy  = "Terraform"
  }
}

# 2. Versioning Configuration
# Protects against accidental deletion and permits object recovery
resource "aws_s3_bucket_versioning" "secure_bucket_versioning" {
  bucket = aws_s3_bucket.secure_bucket.id

  versioning_configuration {
    status = "Enabled"
  }
}

# 3. Server-Side Encryption (SSE)
# Encrypts data at rest using AWS KMS or Amazon S3-managed keys (SSE-S3)
resource "aws_s3_bucket_server_side_encryption_configuration" "secure_bucket_encryption" {
  bucket = aws_s3_bucket.secure_bucket.id

  rule {
    # Apply server-side encryption by default
    apply_server_side_encryption_by_default {
      # Option A: Use S3-Managed Keys (AES256) - Highly cost-effective
      sse_algorithm = "AES256"

      # Option B: Use AWS KMS-Managed Keys (Uncomment below to use custom KMS key)
      # kms_master_key_id = "arn:aws:kms:ap-south-1:123456789012:key/your-custom-kms-key-id"
      # sse_algorithm     = "aws:kms"
    }

    # Minimize AWS KMS API request costs by using S3 Bucket Keys (if KMS is enabled)
    bucket_key_enabled = true
  }
}

# 4. Public Access Block
# Strictly blocks all public access to the bucket and its objects
resource "aws_s3_bucket_public_access_block" "secure_bucket_public_access" {
  bucket = aws_s3_bucket.secure_bucket.id
}
```

- Terraform MCP analyzed the existing Terraform files and is asking whether to overwrite, create, or only display the generated secure S3 Terraform configuration.
- In simple terms, MCP is ready to generate the S3 code, but it wants your approval on where/how to save it.

If the owner (account ID) of the source bucket is the same account used to configure the Terraform AWS Provider, import using the bucket:

```
1 import {
2   to = aws_s3_bucket_server_side_encryption_configuration.example
3   id = "bucket-name"
4 }
```

If the owner (account ID) of the source bucket differs from the account used to configure the Terraform AWS Provider, import using the bucket and expected_bucket_owner separated by a comma (,):

```
1 import {
2   to = aws_s3_bucket_server_side_encryption_configuration.example
3   id = "bucket-name,123456789012"
4 }
```

Using terraform import to import S3 bucket server-side encryption configuration using the bucket or using the bucket and expected_bucket_owner separated by a comma (,). For example:

If the owner (account ID) of the source bucket is the same account used to configure the Terraform AWS Provider, import using the bucket:

```
1 % terraform import aws_s3_bucket_server_side_encryption_configuration.example bucket-name
```

If the owner (account ID) of the source bucket differs from the account used to configure the Terraform AWS Provider, import using the bucket and expected_bucket_owner separated by a comma (,):

```
1 % terraform import aws_s3_bucket_server_side_encryption_configuration.example bucket-name,123456789012
```

```
✓ ReadFile COTASK/provider.tf
✓ ReadFile COTASK/backend.tf
```

Answer Questions

How would you like to receive the secure S3 bucket Terraform code?

- ▲
- 1. Overwrite COTASK/s3.tf
Overwrite COTASK/s3.tf with the secure configuration
- 2. Create new TF file
Create a new separate file in COTASK (e.g., secure_s3.tf)
- 3. Display in chat only
Just display the Terraform code in the chat response
- ▼

Enter to select · [/] to navigate · Esc to cancel

Installation steps

Gemini CLI + Terraform MCP Server

1. Install Prerequisites

```
sudo apt update
sudo apt install -y curl git
```

Check Node.js:

```
node --version
npm --version
```

If Node.js is available, install Gemini CLI:

```
sudo npm install -g @google/gemini-cli@latest
gemini --version
```

2. Install Docker

```
docker --version
```

If Docker is not installed:

```
sudo apt update
sudo apt install -y docker.io
sudo systemctl enable --now docker
```

Verify:

```
docker --version
sudo docker run hello-world
```

3. Pull Terraform MCP Server

```
sudo docker pull hashicorp/terraform-mcp-server

sudo docker images | grep terraform-mcp-server
```

4. Test Terraform MCP Server

```
sudo docker run -i --rm hashicorp/terraform-mcp-server
```

For public Terraform Registry/provider documentation, no TFE_TOKEN is required. Press Ctrl+C after confirming the container starts.

5. Configure Gemini CLI

```
mkdir -p ~/.gemini
nano ~/.gemini/settings.json
```

Add:

```
{
  "mcpServers": {
    "terraform": {
      "command": "docker",
      "args": [
        "run",
        "-i",
        "--rm",
        "hashicorp/terraform-mcp-server"
      ]
    }
  }
}
```

Save with Ctrl+O → Enter → Ctrl+X.

6. Start Gemini

gemini

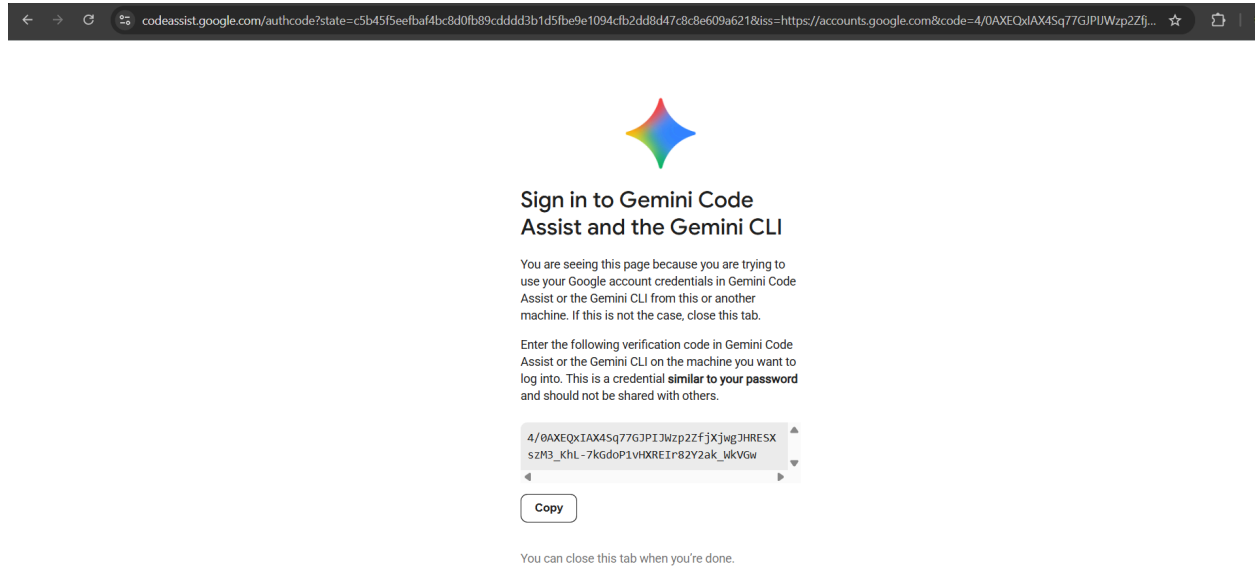
If Gemini asks whether you trust the folder, select Trust folder for your own Terraform project.

7. Authenticate Gemini

Option A: Sign in with Google.

Option B: Use Gemini API Key and enter your Gemini API key. Same all ai toll insatl ther modele

The Gemini API key is for Gemini CLI and is separate from the Terraform MCP server.



8. Verify Terraform MCP

Inside Gemini, use /mcp or ask: List my MCP servers.

gemini mcp list

The expected result should show the Terraform MCP server as connected, for example:
terraform: docker run -i --rm hashicorp/terraform-mcp-server (stdio) - Connected

9. Test Terraform MCP

Inside Gemini, use this prompt:

Create Terraform code for a secure AWS S3 bucket with versioning, encryption, and public access blocked.

10. Create a Terraform Project

```
mkdir -p ~/COTASK  
cd ~/COTASK  
gemini
```

Prompt Gemini:

Create a secure AWS S3 bucket using Terraform. Enable versioning, AES256 encryption, block all public access, enforce HTTPS-only access, and save the configuration as s3.tf.

11. Check the Generated File

```
cd ~/COTASK  
ls -lh  
cat s3.tf
```

12. Run Terraform

```
terraform version  
terraform init  
terraform fmt  
terraform validate  
terraform plan
```

Review the plan before making changes. Only then run:

```
terraform apply
```

Important: Terraform MCP assists the AI with Terraform-specific information and tools. It does not replace the Terraform CLI. Review the generated code and Terraform plan before production changes.

